

Spring Boot Request Lifecycle: Before Controller and After Controller

Complete Interview Material (1–2 Years Experience)

SECTION A – Before Controller (Spring MVC / Web Request Flow)

1) What is Tomcat?

Interview Question

What is Tomcat and why is it important in Spring Boot applications?

Detailed Answer

Tomcat is a web server and servlet container developed by the Apache Software Foundation. In Spring Boot applications, Tomcat is the default embedded server used to run web applications.

When we create a Spring Boot web project using spring-boot-starter-web, Spring Boot automatically includes embedded Tomcat by default.

Why Tomcat is important:

- It listens for HTTP requests from clients (browser, Postman, frontend apps)
- It manages the servlet lifecycle
- It hosts and executes web applications
- It passes incoming requests into the Spring MVC flow

In traditional Spring MVC applications, developers often deploy the application as a WAR file into an external Tomcat server.

In Spring Boot:

- Tomcat is usually embedded
- We run the application directly using main()
- No need to install and configure external Tomcat manually

This makes development easier, faster, and more production-friendly for modern applications.

Simple Explanation

Tomcat means: “The server that runs your Spring Boot web application and receives HTTP requests.”

Example

// When you add this dependency:

spring-boot-starter-web

// Spring Boot automatically includes embedded Tomcat by default.

// Then you can run the application directly:

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Interviewer Cross-Question

Q: Is Tomcat mandatory in Spring Boot?

A: No. Spring Boot can also use Jetty or Undertow, but Tomcat is the default embedded server.

2) What is a Servlet Container?

Interview Question

What is a Servlet Container and what is its role in Java web applications?

Detailed Answer

A Servlet Container is a part of a web server that manages the lifecycle of servlets in a Java web application.

Examples of servlet containers:

- Tomcat
- Jetty
- Undertow

Main responsibilities of a servlet container:

- Creates and initializes servlet objects
- Receives HTTP requests
- Converts them into `HttpServletRequest` and `HttpServletResponse` objects
- Calls the appropriate servlet methods
- Manages servlet lifecycle (init, service, destroy)
- Handles threading and request processing

In Spring MVC / Spring Boot:

- DispatcherServlet is a servlet
- The servlet container (like Tomcat) creates and manages DispatcherServlet
- Then DispatcherServlet handles Spring MVC request routing

So the servlet container is the infrastructure layer that allows Java web applications to work.

Simple Explanation

Servlet Container means: “The component that manages servlets and handles web requests in Java applications.”

Example

Request Flow:

Browser/Postman → Tomcat (Servlet Container) → DispatcherServlet → Controller

Here:

- Tomcat acts as the servlet container
- DispatcherServlet is managed by Tomcat

Interviewer Cross-Question

Q: Is Tomcat only a web server or also a servlet container?

A: Tomcat is both a web server and a servlet container.

3) What is a Servlet?

Interview Question

What is a Servlet in Java and how is it related to Spring Boot?

Detailed Answer

A Servlet is a Java class used to handle HTTP requests and generate HTTP responses in web applications.

Servlets are part of the Java Servlet API and form the foundation of traditional Java web development.

A servlet usually works with:

- HttpServletRequest
- HttpServletResponse

In older Java web applications, developers used to write raw servlets manually to process requests.

In Spring Boot / Spring MVC:

- We usually do not write raw servlets directly
- Spring MVC uses DispatcherServlet internally
- DispatcherServlet is itself a servlet

So even though Spring Boot developers mostly work with controllers, the application still runs on top of the servlet model when using Spring MVC.

This is why servlet basics are important in interviews.

Simple Explanation

Servlet means: “A Java component that handles web requests and responses.”

Example

// Traditional servlet example

```
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
IOException {
        response.getWriter().write("Hello from Servlet");
    }
}
```

// In Spring Boot, we usually do not write this directly.

// Instead, Spring uses DispatcherServlet internally.

Interviewer Cross-Question

Q: Do we usually write raw servlets in Spring Boot?

A: No. In Spring Boot, we typically use controllers, while Spring internally uses DispatcherServlet.

4) What is Spring MVC?

Interview Question

What is Spring MVC and why is it used in Spring Boot web applications?

Detailed Answer

Spring MVC is a web framework inside the Spring ecosystem used to build web applications and REST APIs following the Model-View-Controller pattern.

Spring MVC helps in:

- Handling HTTP requests
- Mapping URLs to controller methods
- Binding request data

- Validating input
- Returning views or JSON responses
- Managing the web request lifecycle

In Spring Boot, when we add:

- spring-boot-starter-web

Spring Boot automatically configures Spring MVC for us.

Spring MVC is commonly used for:

- REST APIs
- Traditional web applications
- Controller-based request handling

Important components in Spring MVC:

- DispatcherServlet
- Controller
- HandlerMapping
- ViewResolver (mainly for traditional MVC)
- Message converters (for REST)

For REST APIs:

- Controllers often use `@RestController`
- Return JSON instead of JSP/HTML views

Simple Explanation

Spring MVC means: “Spring’s web framework that handles requests and routes them to controllers.”

Example

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id) {
        return "User id: " + id;
    }
}
```

Interviewer Cross-Question

Q: Does Spring Boot create Spring MVC automatically?

A: Yes, when we use spring-boot-starter-web, Spring Boot auto-configures Spring MVC.

5) What is MVC architecture?

Interview Question

What is MVC architecture and how does it work in Spring applications?

Detailed Answer

MVC stands for Model-View-Controller. It is a design pattern used to separate responsibilities in web applications.

MVC components:

1. Model

Represents application data and business/domain objects.

Examples:

- DTOs
- Entities
- Form objects

2. View

Represents the UI layer shown to the user.

Examples:

- JSP
- Thymeleaf
- HTML pages

3. Controller

Handles incoming requests, processes input, calls service logic, and returns a view or response.

In traditional Spring MVC:

- Controller returns view names
- ViewResolver resolves JSP/Thymeleaf pages

In REST APIs:

- We still use controller logic from Spring MVC
- But instead of returning views, we often return JSON using `@RestController`

Benefits of MVC:

- Separation of concerns
- Easier maintenance
- Cleaner code structure
- Better testability

Simple Explanation

MVC means: "Separate the app into Model, View, and Controller so each layer has a clear responsibility."

Example

Traditional MVC Flow:

Browser → Controller → Service → Model → View (JSP/Thymeleaf)

REST API Flow:

Client → Controller → Service → Repository → JSON Response

Interviewer Cross-Question

Q: In REST APIs, do we still use MVC?

A: Yes. Spring MVC still handles the request flow, but instead of returning a view, we usually return JSON.

6) What is DispatcherServlet?

Interview Question

What is DispatcherServlet in Spring MVC?

Detailed Answer

DispatcherServlet is the central servlet in Spring MVC that receives all incoming web requests and dispatches them to the correct controller.

It is the heart of Spring MVC.

How it works:

- Request comes from client
- Tomcat receives the request
- Tomcat forwards it to DispatcherServlet
- DispatcherServlet finds the correct controller method
- Executes the controller logic
- Returns response (view or JSON)

Responsibilities of DispatcherServlet:

- Receive incoming requests
- Use HandlerMapping to find matching controller
- Use HandlerAdapter to invoke controller method
- Process exceptions if needed
- Return response through view resolution or message conversion

In Spring Boot:

- DispatcherServlet is auto-configured
- Developers usually do not configure it manually

This is one of the most important Spring MVC interview concepts.

Simple Explanation

DispatcherServlet means: “The main Spring MVC servlet that receives every request and sends it to the correct controller.”

Example

Request Flow:

Browser/Postman → Tomcat → DispatcherServlet → Controller → Response

In Spring Boot, DispatcherServlet is automatically created and registered.

Interviewer Cross-Question

Q: Do we manually create DispatcherServlet in Spring Boot?

A: Usually no. Spring Boot auto-configures and registers it automatically.

7) Why is DispatcherServlet called the Front Controller?

Interview Question

Why is DispatcherServlet called the Front Controller in Spring MVC?

Detailed Answer

DispatcherServlet is called the Front Controller because it acts as a single entry point for all incoming web requests in a Spring MVC application.

Instead of each request directly reaching different servlets or controllers:

- All requests first go to DispatcherServlet
- Then DispatcherServlet decides which controller should handle the request

This is called the Front Controller Pattern.

Why this pattern is useful:

- Centralized request handling
- Common processing in one place
- Easier logging, security, validation, and exception handling
- Cleaner architecture
- Better extensibility

In Spring MVC:

- DispatcherServlet is the front controller
- Controllers are the actual request handlers selected by DispatcherServlet

Simple Explanation

Front Controller means: "One central component receives all requests first, then routes them internally."

Example

Without Front Controller:

Request → Different servlets directly

With Spring MVC:

Request → DispatcherServlet → Correct Controller Method

Interviewer Cross-Question

Q: What is the benefit of Front Controller pattern?

A: It centralizes request handling and makes routing, exception handling, and common processing easier.

8) What happens before a request reaches the controller in Spring Boot?

Interview Question

Explain what happens before a request reaches the controller in Spring Boot.

Detailed Answer

Before a request reaches the controller in Spring Boot, several components are involved in the request processing flow.

Step-by-step flow:

1. Client sends HTTP request

Example:

- Browser
- Postman
- React/Angular frontend

2. Embedded Tomcat receives the request

Tomcat listens on the configured port (usually 8080).

3. Tomcat forwards the request to DispatcherServlet

DispatcherServlet is the main servlet in Spring MVC.

4. DispatcherServlet checks mappings

It looks for the correct controller method using HandlerMapping.

5. HandlerAdapter helps invoke the method

It prepares and calls the controller method properly.

6. Request data is prepared

Spring binds:

- @PathVariable
- @RequestParam
- @RequestBody
- @RequestHeader

7. Then the controller method is executed

So the controller is not the first component. It is part of a larger request lifecycle.

Simple Explanation

Before controller means: "Tomcat and DispatcherServlet process the request first, then Spring routes it to the correct controller."

Example

Full Flow:

Client → Tomcat → DispatcherServlet → HandlerMapping → Controller Method

Interviewer Cross-Question

Q: Is the controller the first component to receive the request?

A: No. Tomcat receives it first, then DispatcherServlet processes it before routing to the controller.

9) What is HandlerMapping?

Interview Question

What is HandlerMapping in Spring MVC?

Detailed Answer

HandlerMapping is a Spring MVC component responsible for finding which controller method should handle an incoming request.

When DispatcherServlet receives a request, it does not directly know which controller method to call.

It asks HandlerMapping:

- Which controller matches this URL?
- Which HTTP method matches?
- Which handler method should be executed?

For example:

If request is:

- GET /users/10

HandlerMapping checks controller annotations like:

- @RequestMapping
- @GetMapping
- @PostMapping

Then it finds the correct method such as:

- getUser(@PathVariable Long id)

So HandlerMapping is responsible for routing resolution inside Spring MVC.

Simple Explanation

HandlerMapping means: "The component that finds which controller method should handle the request."

Example

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id) {
        return "User id: " + id;
    }
}
```

```
// Request: GET /users/10
// HandlerMapping finds the getUser() method
```

Interviewer Cross-Question

Q: Does DispatcherServlet directly know the controller method?

A: No. It uses HandlerMapping to locate the correct handler method.

10) What is the difference between DispatcherServlet and RequestDispatcher?

Interview Question

What is the difference between DispatcherServlet and RequestDispatcher in Java/Spring?

Detailed Answer

DispatcherServlet and RequestDispatcher sound similar, but they are different concepts.

1. DispatcherServlet

- A Spring MVC servlet
- Central front controller in Spring MVC
- Receives all incoming requests
- Routes requests to controller methods

2. RequestDispatcher

- Part of the Java Servlet API
- Used to forward or include requests between server-side resources
- Common in traditional servlet/JSP applications

Example use of RequestDispatcher:

- Forwarding request from one servlet to JSP
- Including content from another resource

Important interview point:

- In Spring MVC, the main request handling component is DispatcherServlet
- RequestDispatcher is not the same thing and is not the main Spring MVC front controller

This distinction is very important because many beginners confuse these two.

Simple Explanation

- DispatcherServlet = Spring MVC front controller
- RequestDispatcher = Servlet API helper used for forward/include

Example

```
// RequestDispatcher example (traditional servlet style)
RequestDispatcher rd = request.getRequestDispatcher("/home.jsp");
rd.forward(request, response);
```

```
// Spring MVC main flow uses DispatcherServlet, not this as the front controller.
```

Interviewer Cross-Question

Q: Which one is the core request entry point in Spring MVC?

A: DispatcherServlet.

SECTION B – After Controller (Service Layer / Repository Flow)

11) What happens after the controller receives the request?

Interview Question

Explain what happens after the controller receives the request in Spring Boot.

Detailed Answer

After the controller receives the request, it usually performs only request-handling responsibilities and then delegates business logic to the service layer.

Typical flow after controller:

1. Request data is already bound

Spring prepares values from:

- @PathVariable
- @RequestParam
- @RequestBody
- @RequestHeader

2. Controller validates or triggers validation

If @Valid is used, request DTO validation happens.

3. Controller calls service layer

The controller should not contain heavy business logic.

4. Service layer executes business logic

Examples:

- Validate business rules
- Fetch entity from repository
- Transform DTO to entity
- Save/update/delete data

5. Service calls repository layer

Repository interacts with the database using JPA.

6. Service returns result to controller

Usually:

- DTO

- Response object
- Boolean/status

7. Controller returns final HTTP response

Often using:

- ResponseEntity
- DTO
- Standard response wrapper

This layered flow improves readability, maintainability, and testability.

Simple Explanation

After controller means: “Controller delegates business work to service, service calls repository, and response comes back to the client.”

Example

```
@PostMapping("/users")
public ResponseEntity<UserResponseDto> createUser(@RequestBody UserRequestDto dto) {
    UserResponseDto response = userService.createUser(dto);
    return ResponseEntity.ok(response);
}
```

Interviewer Cross-Question

Q: Should business logic be written directly in controller?

A: No. Controllers should mainly handle HTTP concerns and delegate business logic to the service layer.

12) What is the role of the Service layer in Spring Boot?

Interview Question

What is the role of the Service layer in Spring Boot applications?

Detailed Answer

The Service layer contains the business logic of the application.

It acts as a bridge between:

- Controller layer
- Repository / DAO layer

The controller should focus on:

- Receiving request
- Validating input

- Returning response

The repository should focus on:

- Database interaction

The service layer should focus on:

- Business rules
- Decision-making
- Data transformation
- Coordinating multiple repository calls
- Transaction boundaries
- Throwing meaningful business exceptions

Examples of logic that belongs in service layer:

- Check whether user exists
- Prevent duplicate email creation
- Validate business constraints
- Map DTO to entity
- Handle create/update/delete workflows

This separation is a key part of clean layered architecture.

Simple Explanation

Service layer means: “The layer where real business logic lives.”

Example

```
public UserResponseDto getUser(Long id) {  
    UserEntity user = userRepository.findById(id)  
        .orElseThrow(() -> new UserNotFoundException("User not found"));  
    return new UserResponseDto(user.getId(), user.getName());  
}
```

Interviewer Cross-Question

Q: Why should we not keep business logic in the controller?

A: Because it makes controllers bulky, harder to test, and mixes HTTP logic with business logic.

13) What is @Service annotation?

Interview Question

What is @Service annotation in Spring Boot?

Detailed Answer

@Service is a Spring stereotype annotation used to mark a class as a service-layer component.

When a class is annotated with @Service:

- Spring detects it during component scanning
- Creates an object (bean) for it
- Manages it in the Spring container
- Allows it to be injected into other classes

Why we use @Service:

- Clearly indicates that the class belongs to the service layer
- Improves readability and architecture clarity
- Allows dependency injection
- Helps in layered design

Technically, @Service behaves similarly to @Component, but it is more semantically meaningful for service-layer classes.

Simple Explanation

@Service means: "This class contains service/business logic and should be managed by Spring."

Example

@Service

```
public class UserService {  
  
    public String getMessage() {  
        return "Service layer working";  
    }  
}
```

Interviewer Cross-Question

Q: Is @Service different from @Component internally?

A: Functionally it behaves similarly for bean registration, but @Service gives better semantic meaning for the service layer.

14) What is Dependency Injection in Spring?

Interview Question

What is Dependency Injection in Spring and why is it important?

Detailed Answer

Dependency Injection (DI) is a design principle where objects receive their required

dependencies from an external framework instead of creating them manually.

In Spring:

- The Spring container creates and manages objects (beans)
- It injects required dependencies automatically

Without DI:

- Classes create dependent objects using new
- Code becomes tightly coupled

With DI:

- Spring injects required objects
- Code becomes loosely coupled
- Easier to test
- Easier to maintain

Example:

- Controller depends on Service
- Service depends on Repository

Spring injects those dependencies rather than developers creating them manually.

This is one of the most important core Spring concepts.

Simple Explanation

Dependency Injection means: “Spring gives the object its required dependencies instead of the class creating them itself.”

Example

// Bad approach (tight coupling)

```
UserService userService = new UserService();
```

// Better approach with Spring

```
@RestController
```

```
public class UserController {
```

```
    private final UserService userService;
```

```
    public UserController(UserService userService) {
```

```
        this.userService = userService;
```

```
    }
```

```
}
```

Interviewer Cross-Question

Q: What is the main benefit of Dependency Injection?

A: Loose coupling, easier testing, and better maintainability.

15) What is constructor injection and why is it preferred over @Autowired field injection?

Interview Question

What is constructor injection and why is it preferred over field injection using @Autowired?

Detailed Answer

Constructor injection is a dependency injection style where dependencies are passed through the class constructor.

Example:

- A service needs a repository
- Spring injects the repository through the constructor

Why constructor injection is preferred:

1. Mandatory dependencies are explicit

If the class cannot work without a dependency, constructor injection makes that clear.

2. Better for immutability

Dependencies can be marked final.

3. Easier unit testing

You can create the class manually in tests by passing mock dependencies.

4. Avoids reflection-based field injection issues

Field injection is less explicit and harder to test.

5. Cleaner and recommended in modern Spring

Field injection using @Autowired still works, but constructor injection is considered best practice.

Since Spring 4.3:

- If a class has a single constructor, @Autowired is optional on the constructor.

Simple Explanation

Constructor injection means: "Inject dependencies through the constructor, which is cleaner and preferred over field injection."

Example

```
@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

Interviewer Cross-Question

Q: Is @Autowired required on constructor injection?

A: If there is only one constructor, Spring can inject it automatically, so @Autowired is usually optional.

16) How do we inject JPA Repository into Service layer?

Interview Question

How do we inject a JPA Repository into the Service layer in Spring Boot?

Detailed Answer

In Spring Boot, JPA repositories are automatically created as Spring beans when we define repository interfaces extending Spring Data JPA interfaces such as JpaRepository.

Example:

- UserRepository extends JpaRepository<UserEntity, Long>

Spring creates a proxy implementation for the repository automatically.

Then we inject that repository into the service layer, usually using constructor injection.

Recommended approach:

- Define repository interface
- Annotate service with @Service
- Inject repository through constructor

This allows the service layer to call:

- findById()
- findAll()
- save()
- deleteById()
- custom query methods

This is a very common real-world interview question.

Simple Explanation

Injecting JPA repository means: “Spring creates the repository bean automatically, and we use constructor injection to use it inside the service.”

Example

```
public interface UserRepository extends JpaRepository<UserEntity, Long> {
}
```

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository userRepository;
```

```
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

Interviewer Cross-Question

Q: Do we need to manually implement JpaRepository methods?

A: No. Spring Data JPA automatically provides implementations for common CRUD methods.

17) What is the role of the Repository layer in Spring Boot?

Interview Question

What is the role of the Repository layer in Spring Boot applications?

Detailed Answer

The Repository layer is responsible for data access and database interaction.

It acts as the layer between:

- Service layer
- Database

Main responsibilities of repository layer:

- Fetch data from database
- Save data
- Update data
- Delete data
- Execute query methods
- Abstract persistence logic

In Spring Data JPA:

- Repository interfaces usually extend JpaRepository or CrudRepository
- Spring generates implementation automatically

Repository should mainly focus on:

- Persistence operations
- Query methods
- Data access logic

Repository should not contain:

- Business rules
- HTTP logic
- Response handling

This separation keeps the application clean and maintainable.

Simple Explanation

Repository layer means: “The layer that talks to the database.”

Example

```
public interface UserRepository extends JpaRepository<UserEntity, Long> {  
  
    Optional<UserEntity> findByEmail(String email);  
}
```

Interviewer Cross-Question

Q: Should business logic be placed in repository?

A: No. Repository should focus on persistence/data access. Business logic should be in the service layer.

18) What is Optional in Java and why is it used with JPA?

Interview Question

What is Optional in Java and why is it commonly used with JPA repository methods?

Detailed Answer

Optional is a container object introduced in Java 8 that may or may not contain a non-null value.

It is used to represent the possibility of absence of a value safely.

Why Optional is useful:

- Avoids direct null handling
- Makes missing values explicit
- Reduces chances of NullPointerException
- Encourages cleaner code

In Spring Data JPA:

- Methods like findById() return Optional<T>

Why?

Because the requested entity may or may not exist in the database.

Example:

- userRepository.findById(10L)

Possible outcomes:

- User exists → Optional contains UserEntity
- User does not exist → Optional is empty

This makes service-layer handling safer and more expressive.

Simple Explanation

Optional means: “A wrapper that may contain a value or may be empty, helping us avoid null-related problems.”

Example

```
Optional<UserEntity> userOptional = userRepository.findById(1L);
```

```
if (userOptional.isPresent()) {  
    UserEntity user = userOptional.get();  
}
```

Interviewer Cross-Question

Q: Why is Optional better than returning null directly?

A: It makes absence explicit and encourages safer handling instead of risky null checks.

19) What is orElseThrow() and why do we use it in service layer?

Interview Question

What is orElseThrow() and why is it commonly used with Optional in the service layer?

Detailed Answer

orElseThrow() is a method of Optional used to return the contained value if present, otherwise throw an exception.

It is very commonly used in the service layer when fetching data from the repository.

Typical use case:

- We call repository.findById(id)
- If entity exists → return it
- If not → throw a meaningful custom exception

Why this is useful:

- Avoids manual if-else checks
- Makes code concise and readable
- Encourages meaningful error handling
- Works well with global exception handling

This is a very common real-world pattern in Spring Boot applications.

Best practice:

Use orElseThrow() with a user-defined exception instead of generic RuntimeException.

Simple Explanation

orElseThrow() means: "Give me the value if present; otherwise throw an exception."

Example

```
UserEntity user = userRepository.findById(id)
    .orElseThrow(() -> new UserNotFoundException("User not found with id: " + id));
```

Interviewer Cross-Question

Q: Why is orElseThrow() preferred over Optional.get()?

A: Optional.get() can throw NoSuchElementException if empty, while orElseThrow() allows us to throw a meaningful custom exception.

20) Why do we throw user-defined exceptions in service layer?

Interview Question

Why do we throw user-defined (custom) exceptions in the service layer instead of generic exceptions?

Detailed Answer

User-defined exceptions (custom exceptions) are exceptions created by developers to represent specific business or application errors clearly.

Examples:

- UserNotFoundException
- DuplicateEmailException
- OrderAlreadyClosedException

Why custom exceptions are better than generic exceptions:

1. Better readability

The exception name itself explains the business problem.

2. Domain-specific meaning

It clearly represents the exact failure case.

3. Easier global exception handling

We can map different exceptions to different HTTP status codes.

4. Cleaner service-layer code

Business rules become more expressive.

5. Better API responses

Clients get meaningful error messages instead of vague generic failures.

In Spring Boot:

- Service layer throws custom exception
- Global exception handler catches it
- Controller returns proper HTTP response

This is a strong sign of mature service-layer design.

Simple Explanation

Custom exception means: “A meaningful exception created for a specific business problem instead of using a generic one.”

Example

```
public class UserNotFoundException extends RuntimeException {
```



```
public UserNotFoundException(String message) {  
    super(message);  
}  
}
```

```
UserEntity user = userRepository.findById(id)  
    .orElseThrow(() -> new UserNotFoundException("User not found with id: " + id));
```

Interviewer Cross-Question

Q: Why not just throw RuntimeException everywhere?

A: Because custom exceptions make code clearer, improve API error handling, and better represent business failures.

21) What is @Transactional and why is it used in service layer?

Interview Question

What is @Transactional and why is it commonly used in the service layer in Spring Boot?

Detailed Answer

@Transactional is a Spring annotation used to define transaction boundaries for database operations.

A transaction means:

- A group of database operations that should succeed together
- If one operation fails, the entire transaction should roll back

Why @Transactional is important:

- Maintains data consistency
- Prevents partial updates
- Ensures atomicity
- Simplifies transaction management

Common use cases:

- Creating parent + child records together
- Updating multiple tables in one business flow
- Complex save/update operations
- Delete + audit logging in one flow

Why it is usually placed in service layer:

- Service layer contains business workflow
- Multiple repository calls often happen there

- Transaction should cover the complete business operation

Important interview point:

- Controller should usually not manage transactions
- Service layer is the preferred place for `@Transactional`

By default:

- `RuntimeException` usually triggers rollback
- Checked exceptions may need explicit configuration depending on use case

Simple Explanation

`@Transactional` means: "Treat this business operation as one database unit—either everything succeeds or everything rolls back."

Example

`@Service`

```
public class UserService {  
  
    private final UserRepository userRepository;  
  
    public UserService(UserRepository userRepository) {  
        this.userRepository = userRepository;  
    }  
  
    @Transactional  
    public UserEntity createUser(UserEntity user) {  
        return userRepository.save(user);  
    }  
}
```

Interviewer Cross-Question

Q: Why is `@Transactional` usually placed on service methods instead of controller methods?

A: Because the service layer defines the business operation and often contains multiple repository calls that should be part of one transaction.

SECTION C – End-to-End Lifecycle / Design Thinking

22) Explain the complete request flow in Spring Boot from browser to database and back

Interview Question

Explain the complete request flow in Spring Boot from browser/client to database and back to the client.

Detailed Answer

The complete request flow in Spring Boot helps us understand how different layers work together from the moment a client sends a request until the response is returned.

Step-by-step complete flow:

1. Client sends HTTP request

The request can come from:

- Browser
- Postman
- React/Angular frontend
- Mobile application

2. Embedded Tomcat receives the request

Tomcat listens on the application port and accepts the incoming HTTP request.

3. Tomcat forwards the request to DispatcherServlet

DispatcherServlet is the central servlet in Spring MVC and acts as the front controller.

4. DispatcherServlet asks HandlerMapping to find the correct controller method

Based on:

- URL path
- HTTP method
- Controller annotations

5. Spring prepares method arguments

It binds:

- @PathVariable
- @RequestParam
- @RequestBody
- @RequestHeader

6. Controller method executes

The controller handles request/response concerns and delegates business logic to the service layer.

7. Service layer executes business logic

Examples:

- Validate business rules
- Call repository methods
- Map DTO to entity
- Throw custom exceptions if needed
- Apply transaction boundaries using `@Transactional`

8. Repository layer interacts with database

Using Spring Data JPA:

- `findById()`
- `save()`
- `findAll()`
- `deleteById()`
- custom queries

9. Database returns result

The repository receives the DB result and returns it to the service layer.

10. Service returns processed result to controller

Usually:

- DTO
- Entity mapped to DTO
- Response object

11. Controller builds HTTP response

Using:

- `ResponseEntity`
- DTO
- Standard response wrapper

12. `DispatcherServlet` sends final response back

For REST APIs:

- Response is converted to JSON by message converters (usually Jackson)

13. Tomcat sends the HTTP response to the client

If an exception occurs:

- Service may throw a custom exception
- Global exception handler catches it
- Proper HTTP error response is returned

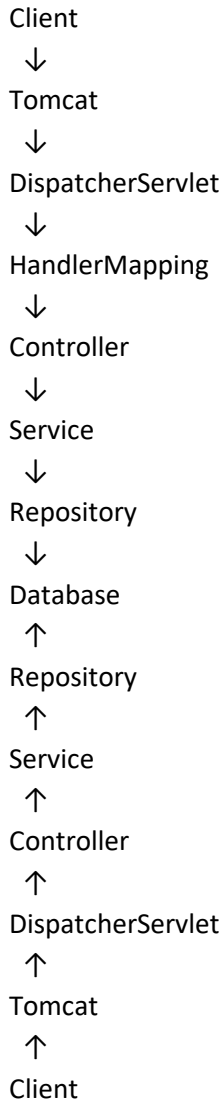
This complete layered flow is one of the most important concepts for interviews.

Simple Explanation

Complete flow means: "Client request goes through Tomcat → DispatcherServlet → Controller → Service → Repository → Database, then the response returns back in reverse order."

Example

Complete Flow:



Interviewer Cross-Question

Q: Where should business logic and transaction management usually be placed?

A: In the service layer, because it represents the full business operation and coordinates repository calls.

23) Why do we separate Controller, Service, and Repository layers?

Interview Question

Why do we separate Controller, Service, and Repository layers in Spring Boot applications?

Detailed Answer

Separating Controller, Service, and Repository layers is a core design principle in Spring Boot because it follows separation of concerns.

Each layer has a specific responsibility:

1. Controller Layer

- Handles HTTP request and response
- Reads path params, query params, request body
- Returns response to client

2. Service Layer

- Contains business logic
- Applies business rules
- Coordinates workflows
- Handles transactions
- Throws meaningful business exceptions

3. Repository Layer

- Handles database operations
- Performs CRUD and query execution
- Interacts with JPA / persistence layer

Why separation is important:

- Cleaner architecture

Each class has one clear responsibility.

- Easier maintenance

Changes in one layer affect fewer parts of the system.

- Better testability

Each layer can be tested independently.

- Reusability

Service logic can be reused across multiple controllers.

- Better readability

Developers can quickly understand where logic belongs.

- Scalable design

Large projects become easier to manage.

Without proper separation:

- Controllers become huge
- Business logic gets mixed with HTTP logic
- Database code becomes scattered
- Code becomes hard to maintain

Simple Explanation

Layer separation means: “Each layer should do only its own job, which makes the application cleaner and easier to maintain.”

Example

Controller → handles request/response

Service → handles business logic

Repository → handles database access

Example:

UserController → UserService → UserRepository

Interviewer Cross-Question

Q: What happens if we put business logic directly in the controller?

A: Controllers become bulky, harder to test, and the code violates separation of concerns.

24) Where should validation, business logic, and exception handling be placed?

Interview Question

Where should validation, business logic, and exception handling be placed in a Spring Boot layered application?

Detailed Answer

In a layered Spring Boot application, each type of logic should be placed in the appropriate layer to keep the design clean and maintainable.

1. Validation

Validation can happen in two places:

A. Request/Input validation (Controller layer)

Use:

- @Valid

- Bean Validation annotations like @NotBlank, @Email, @Size

This validates incoming request DTOs before business logic starts.

B. Business validation (Service layer)

Examples:

- Email must be unique
- User cannot be deleted if active orders exist
- Status transition rules

These are business rules, so they belong in the service layer.

2. Business Logic

Business logic should be placed in:

- Service layer

Examples:

- Create/update workflows
- Decision-making
- Multi-step DB operations
- Rule enforcement
- Calling multiple repositories
- Transaction boundaries

3. Exception Handling

Exception throwing:

- Often in service layer (custom exceptions for business problems)

Exception response handling:

- Preferably in a global exception handler using:
 - @ControllerAdvice
 - @RestControllerAdvice
 - @ExceptionHandler

Why this structure is best:

- Controller remains thin
- Service remains business-focused
- Error handling stays centralized
- API responses stay consistent

This is a very strong real-world architecture answer in interviews.

Simple Explanation

Best practice is: "Input validation in controller, business validation and logic in service, and centralized exception handling in a global exception handler."

Example

```
// Controller: input validation
@PostMapping("/users")
public ResponseEntity<UserResponseDto> createUser(@Valid @RequestBody UserRequestDto
dto) {
    return ResponseEntity.ok(userService.createUser(dto));
}

// Service: business validation
if (userRepository.findByEmail(dto.getEmail()).isPresent()) {
    throw new DuplicateEmailException("Email already exists");
}

// Global exception handler
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity<String> handleDuplicate(DuplicateEmailException ex) {
        return ResponseEntity.badRequest().body(ex.getMessage());
    }
}
```

Interviewer Cross-Question

Q: Should all validation be done only in the controller?

A: No. Basic input validation can happen in the controller, but business-rule validation should be done in the service layer.